Beat the Bell: Class-Schedule-Aware Stockout Prediction and Pre-emptive YouBike Rebalancing for Campus Stations

Midterm Report (HW3) · Introduction to Artificial Intelligence (TAICA, Fall 2026) · Group 54 · *blue text = new or changed since the proposal (HW1)*

Team members

國立中山大學 B124020041 林沛霆 · 國立中山大學 B134020002 陳佳晨 · 國立中山大學 B134020008 林詩穎

Abstract

Docks near a university empty within minutes of a class ending. Because the timetable makes this surge predictable, we cast rebalancing as an AI decision problem: one truck moves bikes before the bell instead of after the stockout. We (i) reconstruct censored demand and the operator's actual truck moves from public trip records and inventory snapshots, (ii) learn a schedule-aware, calibrated predictor of P(stockout within h min) per station, and (iii) plan truck routes that minimize expected unmet rentals plus travel cost. This midterm report delivers the first three stages on real data. A logger has recorded all 309 YouBike stations within 5 km of NSYSU every 10 minutes since 1 October; at the eight campus stations bikes fall from 193 at 10:00 to 7 at 18:00 on teaching days, and 55% of the campus stock leaves within 20 minutes of the 12:00 bell. On 10 months of Taipei trip records and 295 days of 10-minute inventory for 144 stations around NTU, our reconciliation recovers the operator's truck moves (R^2 = 0.98 for reproducing inventories) and shows that observed rentals understate demand by up to 2× at campus stations in the evening. A gradient-boosting predictor reaches AUC 0.93, Brier 0.080 and ECE 0.012 at a 30-minute horizon on three held-out months, and transfers to NSYSU zero-shot (AUC 0.93 on 309 stations, 0.94 on campus). A discrete-event replay simulator with greedy truck policies avoids 8%, 17% and 27% of unmet rentals with 1, 3 and 6 trucks; the probabilistic policy also cuts empty station-minutes by 41%. The remaining work is the route-search planner and the live NSYSU demo.

1. Introduction

1.1. Motivation

Taipei's YouBike 2.0 alone served 46 million trips in 2023 [3]. Around universities it fails in one specific way: bikes are plentiful during lectures and vanish when class ends. Fig. 1(a) quantifies this for the 59 NTU campus stations over 20 weekdays [2]: the campus count drops from 584 to 392 bikes within 20 minutes of the 12:10 bell and by 41% after the 17:20 bell; after 18:00 about 30% of stations are empty. Fig. 1(b) is our campus: twelve minutes after NSYSU's 17:00 bell on 28 Sep 2026, 4 of the 25 stations within 1.5 km were empty, all on the KRT side, while the gymnasium held 48 bikes. The bikes exist; they stand at the wrong stations at a moment the timetable predicts. Two weeks of our own logging confirm that the snapshot was typical rather than unlucky: on eight teaching weekdays the eight campus stations (231 docks) averaged 193 bikes at 10:00, 49 at 17:00 and 7 at 18:00, with 64% of them empty at 18:00, while the two KRT Sizihwan exits were empty in 36% of morning slots and held 51 bikes at 18:00 (Sec. 3.2). The problem is a tide between campus and the metro, in both directions, every teaching day.

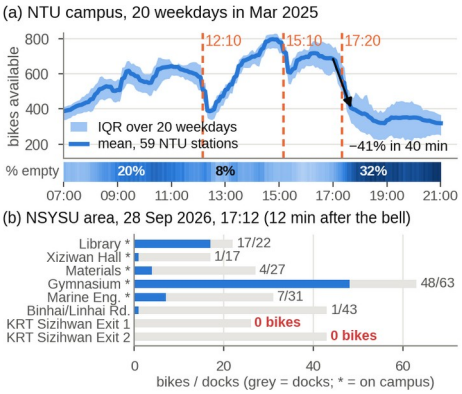


Figure 1. Class-end stockouts are real and predictable (from the proposal). (a) Mean bikes available at 59 NTU campus stations over 20 weekdays, Mar 2025 [2]; orange = period ends; strip = share of empty stations. (b) NSYSU at 17:12, 28 Sep 2026 [12].

1.2. Problem definition

Station i has capacity c_i and inventory $s_i(t)$; rentals $r_i(t)$ and returns $q_i(t)$ are counts whose rates depend on time of day, day of week and, our key assumption, the time to the next bell $b(t)$. Rentals are censored: when $s_i(t) = 0$, lost rentals are unrecorded. One truck of capacity Q visits stations with travel times d_{ij}. Before each bell we choose a route and load plan π minimizing expected unmet rentals over a horizon H plus β·travel time under capacity and deadline constraints. This is the course's rational-agent view: acting under uncertainty to maximize expected utility, coupling probabilistic reasoning, learning and search. Two sub-problems are now defined precisely. *Stockout prediction:* for every station and 10-minute slot t, output $p_i(t, h) = P(\min_{\tau\in(t,t+h]} s_i(\tau) = 0)$ for $h \in \{30, 60\}$ min. *Rebalancing:* with K trucks, choose moves (source, target, k bikes) that maximize the expected rentals saved per truck-minute, evaluated by replaying the real trips of held-out days.

1.3. Existing solutions and literature

Static repositioning is a routing-and-inventory problem [4]; dynamic repositioning combines forecasts with heuristics or mixed-integer programs [5, 6]; recent work uses spatio-temporal reinforcement learning [7]. Two ingredients matter on a campus yet are rarely combined: censored-demand estimation by simulation [8] or censored Gaussian processes [9], and inferring the operator's unpublished truck moves by subtracting trip flows from inventory changes [10]; on Taipei data this reveals 18–20 thousand unexplained bike moves on 12 Mar 2025, peaking at 07–09 and 18–20. A 2025 NTU thesis optimized dispatch near NTU with survival-model demand on one month of trips [11]. No published work, to our knowledge, (a) uses the class timetable as an explicit signal, (b) outputs calibrated stockout probabilities rather than point forecasts, or (c) tests transfer between campuses; NSYSU has never been studied and Kaohsiung publishes no trip data, so transfer is the practical question. For the predictor we use gradient-boosted decision trees [13], which handle the mixed count and categorical features natively, and we measure probability quality with the Brier score [15] and the expected calibration error [14] rather than accuracy, because the planner consumes probabilities.

1.4. Proposed solution

Fig. 2 shows the pipeline. Open NTU data (hourly trips [1]; 10-minute inventory over 416 days [2]) serve to reconstruct demand and operator moves, to train a schedule-aware gradient-boosting predictor with reliability checks, and to drive a rolling-horizon route planner; policies are compared in a replay simulator fed with real trips. The predictor is then ported to NSYSU with a few weeks of locally logged inventory [12] for a live web demo that recommends a pre-bell truck plan. Contributions: (1) an uncertainty-aware pre-emptive rebalancing method reproducible from open data; (2) the first quantification of class-end stockouts at NTU and NSYSU; (3) an NTU→NSYSU transfer study. Status at midterm: (2) is complete for both campuses (Secs. 3.2–3.3); (1) is implemented end to end with a greedy planner and evaluated in the simulator (Sec. 3.6); (3) is complete for the predictor (Sec. 3.5). Code, data tables and results are public [17].

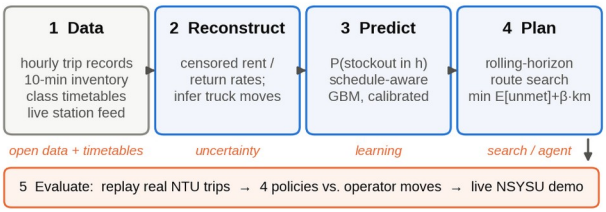


Figure 2. Proposed pipeline and the course unit each stage exercises (from the proposal).

2. Method

2.1. System overview

Fig. 3 shows the system as implemented. The NTU lane turns two open data sets into three tables (hourly flows, a 10-minute inventory panel and an hourly reconciliation table) that feed demand reconstruction, the predictor and the simulator; the NSYSU lane produces the same inventory panel from our logger, so the inventory-only predictor and the planner run unchanged on either campus. Every module is a short Python script in the repository [17] and every table can be regenerated from the raw sources with the commands in its README.

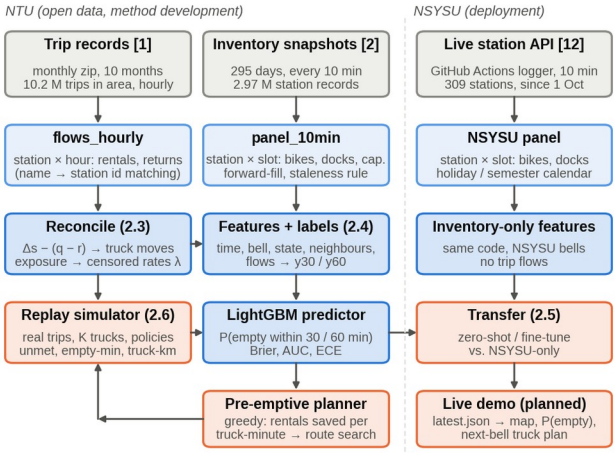


Figure 3. Architecture as implemented: sources (grey), tables (light blue), learning modules (blue) and decision or evaluation modules (orange). Section numbers refer to this report.

2.2. Data acquisition and panel construction

NSYSU logger. YouBike publishes a JSON of every station (bikes, free docks, status, update time) but no history. A GitHub Actions workflow fetches it every 10 minutes for the 309 stations within 5 km of NSYSU and commits one CSV row per station and slot. Because GitHub throttles cron jobs in quiet repositories (4 runs in our first day), one run now logs for 340 minutes and dispatches its successor before ending; an hourly cron only restarts the chain if it breaks, and a guard script ensures a single active run. Since 2 Oct the logger has delivered 144 snapshots per day with one 6-hour gap (12 Oct). Station status 2 (out of service) is stored so that an offline station is not mistaken for a stockout.

NTU trips. The Taipei trip records [1] give rent time, rent station, return time, return station and duration for every ride, with times truncated to the hour and stations named rather than numbered; 2024 files have six columns and 2025 files seven. We stream each monthly zip (6–7 M rows) in chunks, keep rides with either end among the 144 stations within 1.2 km of the NTU campus centroid (59 on campus, 85 in the surrounding ring), match names to station ids through the station table of [2], and aggregate

rentals $r_i(h)$ and returns $q_i(h)$ per station and hour. Ten months (Sep 2024–Jun 2025, 10.2 M rides in the area) take 15 s per month.

NTU inventory panel. The snapshot repository [2] stores one row per station update (about 95 per station per day). Stations report only when something changes, so a quiet night produces no rows while a daytime silence of hours means the station is offline; we forward-fill onto a 10-minute grid and set the value to unknown when the last report is older than 120 min during 07:00–23:00 or 600 min otherwise. The result is 6.1 M station-slots with 7.9% unknown, concentrated in a collection outage on 1–16 Oct 2024.

Calendars. Bell times are the period ends of each campus (NTU: 09:00, 10:00, 11:10, 12:10, 13:10, 14:10, 15:10, 16:20, 17:20, 18:20, 19:15, 20:10; NSYSU: on the hour from 10:00 to 18:00, then 19:10, 20:10, 21:10). Taiwanese public holidays and the two NTU semesters (2 Sep 2024–10 Jan 2025, 17 Feb–13 Jun 2025) define the off-day and semester-week features.

2.3. Demand reconstruction and operator inference

Per station-hour, $\Delta s_i = q_i - r_i + m_i + \varepsilon$, with m_i the operator's net truck move; residuals $|\Delta s - (q - r)| \geq 5$ over 3-hour windows are labelled as moves [10], giving an observed operator policy. For censoring, minutes with $s_i = 0$ are dropped from the exposure of a Poisson rate model, so λ_r reflects only time when a rental was possible [8, 9]. Implementation: the stock at the start of each hour is the grid value at hh:00, the exposure is the share of the hour's six slots with $s_i > 0$, and $\lambda_r(i,$ day type, hour$) = \Sigma$ rentals / Σ exposure. Two validity checks are reported in Sec. 3.3: the R^2 of Δs against $(q - r)$ on hours without inferred moves, which measures how well trips alone explain inventory, and the R^2 for reproducing the next-hour stock when the inferred moves are added, which the proposal required to exceed 0.8 before the inferred policy may be used as a benchmark.

2.4. Schedule-aware stockout prediction

Features per station and 10-minute slot: minutes to the next and since the last bell, day of week, week of semester, lagged flows, current stock, capacity and stock at the nearest stations. Baselines: persistence and seasonal mean; then gradient boosting with reliability checks. The implemented predictor is a direct classifier of the label $y_h = 1$ if the station is empty at any slot in $(t, t + h]$, $h \in \{30, 60\}$ min, built for 06:00–23:00. The 22 inventory-only features are: time (hour of day, day of week, off-day flag, semester week, in-semester flag); bell (minutes to the next and since the last period end, and the same for the three major bells 12:10, 15:10, 17:20); state (bikes, free docks, capacity, fill ratio, change over the last 10, 30 and 60 min, minimum over the last hour, empty-now flag, minutes empty in the last hour); neighbours (bikes and empty count at the 3 nearest stations, mean fill of the 5 nearest). The full NTU variant adds six flow features (rentals and returns in the last two complete hours, rentals in the same hour last week, mean rentals for this station and hour-of-week on the training period) and the station id as a categorical feature. Models are LightGBM [13] binary classifiers (63 leaves, learning rate 0.05, 80% feature and row subsampling, L2 = 1) trained on a 40% random subsample of the training slots with early stopping on the last three training weeks. Baselines are persistence ($p = 1$ if the station is empty now) and the seasonal mean of y_h per station, day type and slot. Metrics: Brier score (mean squared error of p) [15], AUC and ECE with ten equal-width bins [14]; reliability curves are reported because the planner multiplies p by demand.

2.5. Transfer to NSYSU

Kaohsiung publishes no trip data, so only the inventory-only variant can run at NSYSU. The NSYSU panel is built from the logger with NSYSU bells and calendar, and the same feature code is applied. We compare, on six held-out days (10–15 Oct), (a) persistence, (b) a seasonal baseline fitted on the six training days (2–7 Oct), (c) the NTU model applied zero-shot, (d) the NTU model fine-tuned by continuing boosting on the NSYSU training days (learning rate 0.02, early stopping on 8–9 Oct), and (e) a model trained on NSYSU days only. Results are reported for all 309 stations, the 19 stations of the campus–KRT corridor and the 8 campus stations.

2.6. Pre-emptive planner and replay simulator

State: truck position and load, station stocks and predicted rates; actions: (station, $\pm k$ bikes); objective $J(\pi) = \Sigma_i \, E[\max(0, D_i - s_i)] + \beta \cdot T(\pi)$, with D_i the predicted demand. A discrete-event replay simulator consumes real held-out trips: a rental is served only if stock > 0, otherwise it counts as unmet, while the truck follows the policy under test. We compare none, reactive, predictive-point and predictive-probabilistic policies, plus the inferred operator moves. Metrics: unmet rentals, empty station-minutes, truck-km. Implemented simulator. Each day runs from 06:00 to 23:00 with the initial stock taken from the 06:00 snapshot. Recorded rentals at the 144 stations are placed uniformly at random within their recorded hour (seeded); a served rental generates the return of that ride after its recorded duration, an unserved one disappears; rides from outside the area arrive as recorded. A full station redirects a return to the nearest station with a free dock. Trucks ($K \in \{1, 3, 6\}$, capacity 20) travel at 15 km/h on 1.3× straight-line distance with 3 min per stop and 6 s per bike. Every 10 minutes each idle truck receives one move (source, target, k) chosen greedily to maximize the expected rentals saved per truck-minute: the benefit of a move is $v_{\text{target}} \cdot \min(k,$

λ_{target}) and its cost is travel plus handling time. The three policies differ only in how targets and the weight v are defined: reactive serves stations that are already empty (v = 1); predictive-point serves stations whose next-hour demand exceeds stock, $\lambda - s \geq 2$ (v = 1); predictive-probabilistic serves stations with $p_i(t, 60) \geq 0.3$, weights them by $v = p_i$ and never takes bikes from a station with $p_i > 0.2$, where p is computed on the simulated inventory by the inventory-only predictor at every decision. Sources are stations whose stock exceeds their next-hour demand by at least 5 bikes. The operator reference injects the inferred hourly moves of Sec. 2.3 directly, i.e. it replays what actually happened to the inventory. The rolling-horizon route search with 2-opt and simulated annealing [16] is the next step; the greedy loop is its initial solution.

2.7. Live demo

The logger already writes latest.json, a compact snapshot of the 309 stations, on every run. The demo page will read it, compute the inventory-only features in the browser or a small Python service, show P(empty within 30 min) per station on a map and the next-bell truck plan of the planner, as mocked in Fig. 11.

3. Experiments

3.1. Datasets, splits and environment

Table 1 lists the data. NTU rows are split by time: training 1 Sep 2024–9 Mar 2025, validation 10–31 Mar 2025 (early stopping), test 1 Apr–22 Jun 2025 (1.22 M station-slots, three months never seen by the model). NSYSU uses 2–7 Oct for training, 8–9 Oct for validation and 10–15 Oct for testing. The replay simulator uses the 28 in-semester weekdays of the test period with ≥ 95% snapshot coverage (1 Apr–13 Jun 2025). Everything runs on a 2-vCPU, 7 GB container with Python 3.11, pandas 3.0, LightGBM 4.7 and scikit-learn 1.8; building the 4.3 M-row feature table takes 21 s, training all four NTU models 5 min, and one simulated day with five policies about 3 s.

Table 1. Data used in this report.

Source	Coverage / resolution	Rows used
Taipei YouBike 2.0 trip records [1]	Sep 2024–Jun 2025, hourly, 1,519 stations	10.2 M rides touching the 144 NTU-area stations
Taipei inventory snapshots [2]	1 Sep 2024–22 Jun 2025 (295 days), 10 min	2.97 M station records → 6.1 M station-slots
NSYSU logger [17]	1–15 Oct 2026, 10 min, 309 stations ≤ 5 km	614 k station-slots (14 complete days)
Calendars	period ends of both campuses, holidays, semesters	bell and off-day features

3.2. Class-end stockouts at NSYSU

Fig. 4 summarizes the eight teaching weekdays of 2–14 Oct 2026 (9–10 Oct are holidays). The campus and the KRT exits are the two ends of a tide: campus stations fill between 08:00 and 10:00 (193 of 231 docks occupied) while both KRT exits are empty in 36% of morning slots; after the 12:00 bell the campus loses 93 bikes (55%) in 20 minutes, recovers over lunch, and then drains through the 16:00 and 17:00 bells (−68 and −47 bikes within 40 minutes) to 7 bikes at 18:00, when 64% of campus stations are empty and the KRT exits hold 51 bikes. Fig. 5 aligns the drops on the bell: the 17:00 bell removes 88% of what is left. The 246 stockout episodes at campus stations amount to 3.8 per station and day, with a median length of 20 min, a mean of 30 min and a 90th percentile of 60 min; in total the eight campus stations spend 934 station-minutes empty per weekday. The morning has the mirror problem: the engineering and library stations are full (no free dock) in 44% and 25% of 08:00–11:00 slots. On weekends the campus holds 40–130 bikes all day and the pattern disappears, which confirms the timetable as the cause.

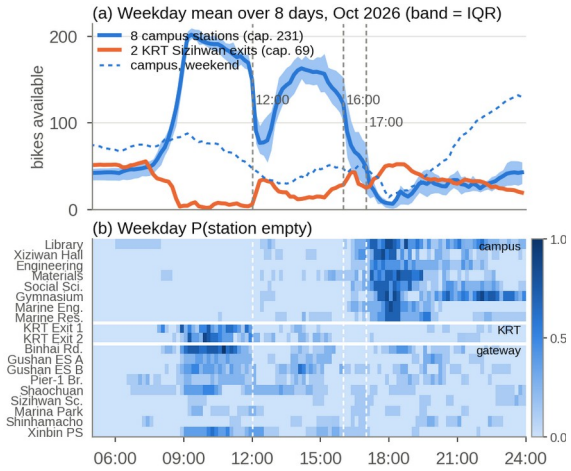


Figure 4. NSYSU, eight teaching weekdays in Oct 2026 from our logger. (a) Bikes available at the 8 campus stations and the 2 KRT Sizihwan exits (mean, band = IQR); dashed = weekend campus mean. (b) Probability that a station is empty, by station and time of day.

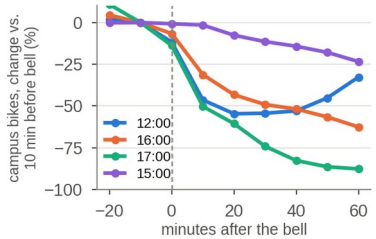


Figure 5. Campus stock relative to 10 minutes before the bell, NSYSU weekdays.

3.3. Demand reconstruction and operator moves at NTU

Reconciliation covers 917 k station-hours (144 stations, 295 days). On hours without inferred moves, trips alone explain inventory changes with $R^2 = 0.87$; adding the inferred moves reproduces the next-hour stock with $R^2 = 0.984$ (0.63 without them), so the inferred operator policy passes the proposal's threshold. The operator moves about 2,550 bikes into and 2,160 bikes out of the area per day, 10% of the 25,700 daily rentals. Fig. 6 shows a reactive pattern: pick-ups from campus peak at 08:00–09:00 after the morning inflow, drop-offs at campus peak at 16:00–17:00 and the ring is emptied at 17:00; only 19% of campus drop-offs happen in the hour before a major bell. Fig. 7 shows the censoring correction for campus stations on weekdays: observed rentals understate demand by 8% at 12:00, 17% at 17:00, 39% at 18:00 and 2.1× at 22:00, because campus stations are empty 46% of the time at 17:00–19:00.

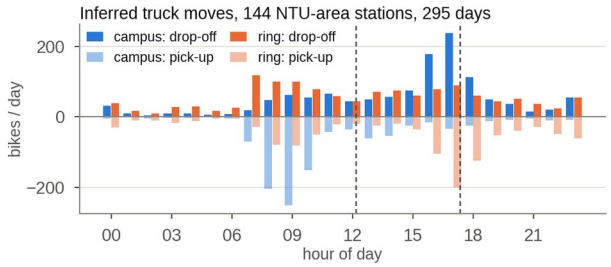


Figure 6. Inferred truck moves per hour and day at the 59 campus and 85 ring stations (bikes dropped off above the axis, picked up below). Dashed lines: 12:10 and 17:20 bells.

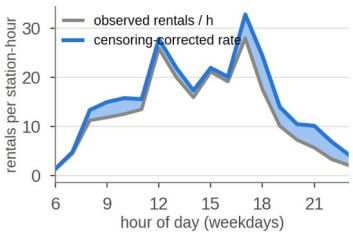


Figure 7. Observed and censoring-corrected rental rates, campus stations, weekdays (spring 2025).

3.4. Stockout prediction

Table 2 and Fig. 8 give the test results. At the 30-minute horizon the full model reaches AUC 0.931 and Brier 0.080, against 0.818 / 0.126 for the seasonal baseline and 0.742 / 0.125 for persistence; the inventory-only model is close behind (0.921 / 0.084), so trip flows add little once the current stock and its trend are known. Calibration is good without post-processing: ECE 0.012 and a reliability curve on the diagonal up to $p = 0.9$ (Fig. 8a), while the seasonal baseline under-predicts at high probabilities. Performance is higher on campus stations (AUC 0.945) and lower in the 40 minutes after a major bell (0.905), where the surge is fastest. At 60 minutes AUC drops to 0.916 (full) and 0.898 (inventory-only). By gain, the current stock (49%), its one-hour minimum (20%), the station id (8%) and the fill ratio (5%) dominate; the bell features contribute almost nothing, and an ablation without them changes AUC by less than 0.002 on every subset, including the campus slots 0–30 min before a bell. Hour of day already encodes a fixed timetable; the explicit bell features will matter only if they help the planner look further ahead than the stock trend does. The hard cases are stations that are still well stocked 30 minutes before a bell: AUC 0.79 at $h = 30$, which is the target for the second half (longer horizons, demand-rate features from Sec. 2.3).

Table 2. NTU test period (Apr–Jun 2025), all 144 stations. GBM + flows also uses the station id. ECE = expected calibration error.

Model	h	Brier	AUC	ECE	trees
persistence	30	0.125	0.742	0.125	–
seasonal mean	30	0.126	0.818	0.016	–
GBM inventory only	30	0.084	0.921	0.013	170
GBM + flows	30	0.080	0.931	0.012	951
persistence	60	0.178	0.698	0.178	–
seasonal mean	60	0.149	0.816	0.017	–
GBM inventory only	60	0.109	0.898	0.013	138
GBM + flows	60	0.101	0.916	0.012	225

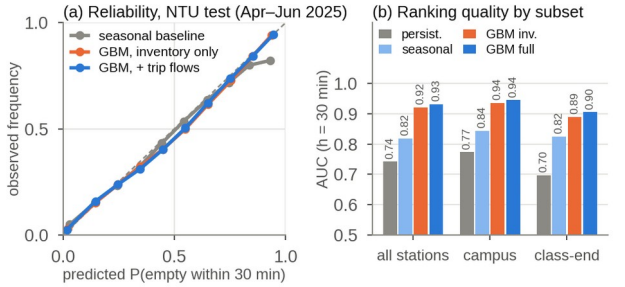


Figure 8. (a) Reliability of the 30-minute predictions on the NTU test months. (b) AUC on all stations, campus stations and campus slots within 40 min after a major bell.

3.5. Transfer to NSYSU

Fig. 9 and Table 3 show that the inventory-only NTU model works at NSYSU without any local data: zero-shot AUC 0.934 on all 309 Kaohsiung stations and 0.944 on the 8 campus stations at $h = 30$, with Brier 0.049 / 0.068 and ECE 0.022 / 0.020. Fine-tuning on six NSYSU days adds 0.006 AUC and improves ECE to 0.003 on all stations; a model trained only on those six days is equally good (0.940), while the seasonal baseline from six days is poor (0.651). The proposal expected a zero-shot drop that local data would close; instead the dynamics of a station emptying generalize across cities almost perfectly, and the only transfer gap is in calibration, which six days of local data remove. At $h = 60$ the figures are 0.892 zero-shot and 0.906 fine-tuned.

Table 3. NSYSU test days (10–15 Oct 2026), h = 30 min; AUC / Brier.

Model	309 stations ≤ 5 km	8 campus stations
persistence	0.738 / 0.061	0.742 / 0.104
seasonal, 6 NSYSU days	0.651 / 0.096	0.789 / 0.129
NTU model, zero-shot	0.934 / 0.049	0.944 / 0.068
NTU model + fine-tune	0.940 / 0.047	0.945 / 0.069
NSYSU-only, 6 days	0.940 / 0.047	0.942 / 0.069

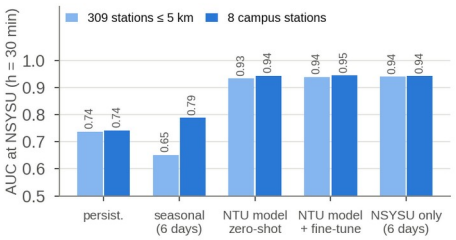


Figure 9. AUC at NSYSU, h = 30 min, for the five models of Sec. 2.5.

3.6. Replay simulation

Over the 28 replayed weekdays the area records 32,100 rentals per day; without rebalancing 3,390 of them (10.5%) find an empty station and the 144 stations spend 17,800 station-minutes empty per day. Table 4 and Fig. 10 give the policies. One truck avoids 5–8% of unmet rentals with 55–67 km of driving; three trucks 16–17%; six trucks 25–27%. The probabilistic policy is best on every fleet size and the point-forecast policy worst, but the margins on unmet rentals are small (27% vs. 26% vs. 25% with six trucks); the clearest difference is in empty station-minutes, which the probabilistic and reactive policies cut by 41% with six trucks against 14% for the point policy, because the latter waits for a forecast deficit instead of acting on an empty station. Replaying the inferred operator moves, which inject about 5,100 bikes per day without any truck constraint, avoids 61%; it is an upper reference, not a policy. The ranking shows that at this stage the forecast is not the bottleneck: the greedy loop serves one target per truck at a time, cannot combine a pick-up with several drop-offs, and ignores the deadline structure of the bells. The rolling-horizon route search planned for November attacks exactly this, and the simulator now gives it a fixed benchmark.

Table 4. Replay of 28 NTU weekdays (Apr–Jun 2025), per-day means; empty-min = station-minutes with no bike.

Policy	trucks	unmet	empty-min	moved	km
none	–	3,389	17,793	0	0
reactive	1	3,139	16,300	842	65
point forecast	1	3,209	17,280	944	55
probabilistic	1	3,135	16,527	883	67
reactive	3	2,820	13,743	1,995	214
point forecast	3	2,863	16,450	2,374	192
probabilistic	3	2,806	13,860	2,139	223
reactive	6	2,496	10,503	2,876	425
point forecast	6	2,543	15,269	3,566	396
probabilistic	6	2,489	10,529	3,166	456
operator replay	–	1,337	9,448	5,121	–

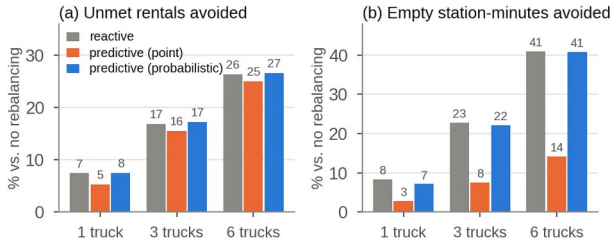


Figure 10. Reduction of (a) unmet rentals and (b) empty station-minutes relative to no rebalancing, by policy and fleet size.

3.7. Progress, risks and next steps

Fig. 12 updates the schedule: data pipeline and demand reconstruction are complete, the predictor is trained and transferred ahead of plan, and the simulator exists two weeks early, with the planner still greedy. Of the proposal's three fallbacks, none was needed: Kaohsiung history is being collected by our own logger, the probabilistic policy does beat the point forecast, and local search is on schedule. Remaining work: (i) the rolling-horizon planner with 2-opt / simulated annealing and multi-stop routes, evaluated in the same simulator; (ii) longer horizons and demand-rate features for the pre-bell cases where the predictor is weakest; (iii) imputing the censored demand of Sec. 3.3 into the replay so that unmet rentals include the riders who never found a bike; (iv) the live NSYSU demo (Fig. 11) and, after four more weeks of logging, a transfer study by week of local data.

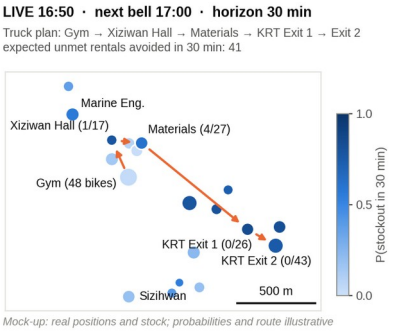


Figure 11. Mock-up of the live NSYSU demo (from the proposal; real station positions and stock, probabilities and route illustrative).

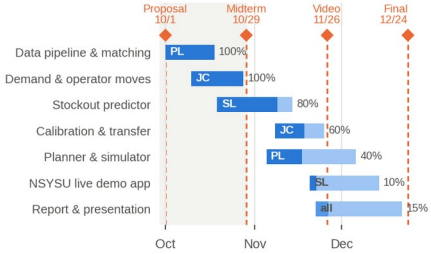


Figure 12. Schedule with completion at 29 Oct (dark = done; PL 林沛霆, JC 陳佳晨, SL 林詩穎).

3.8. Work division and reproducibility

林沛霆 (PL) built the logger, the NTU pipeline, the replay simulator and this report; 陳佳晨 (JC) owns demand reconstruction, operator inference and the calibration and transfer studies; 林詩穎 (SL) owns the predictor, the ablations and the demo. All code, the reconciliation tables, model files, metrics and figures are in the public repository [17] under analysis/; the README documents how to rebuild every table from the raw sources, and the logger keeps adding NSYSU data every 10 minutes.

References

[1] 臺北市政府交通局. 臺北市公共自行車 2.0 租借紀錄, 2020–2026, data.taipei.

[2] C.-X. Jian. youbike-historical-data (10-min Taipei YouBike 2.0 inventory snapshots), GitHub, 2024–2025.

[3] 報導者 The Reporter. 台北市 YouBike 多熱門？數據告訴你, Apr. 2024.

[4] T. Raviv et al. Static repositioning in a bike-sharing system: models and solution approaches. EURO J. Transp. Logist., 2(3):187–229, 2013.

[5] J. Liu et al. Rebalancing bike sharing systems: A multi-source data smart optimization. In KDD, 2016.

[6] S. Ghosh et al. Dynamic repositioning to reduce lost demand in bike sharing systems. J. Artif. Intell. Res., 58:387–430, 2017.

[7] Y. Li, Y. Zheng, and Q. Yang. Dynamic bike reposition: A spatio-temporal reinforcement learning approach. In KDD, 2018.

[8] A. Negahban. Simulation-based estimation of the real demand in bike-sharing systems in the presence of censoring. Eur. J. Oper. Res., 277(1):317–332, 2019.

[9] D. Gammelli et al. Estimating latent demand of shared mobility through censored Gaussian processes. Transp. Res. Part C, 120:102775, 2020.

[10] H. Luo et al. Estimating rebalancing activities and vehicle use for existing station-based bike share systems. Transportation, 2026.

[11] 周欣儀. 臺大校總區周遭 YouBike 2.0 使用行為與調度優化研究. 碩士論文, 國立臺灣大學機械工程學系, 2025.

[12] YouBike 即時站點資料 (apis.youbike.com.tw); 交通部 TDX (tdx.transportdata.tw); 高雄市開放資料 kcgoa-00000595-e5c.

[13] G. Ke et al. LightGBM: A highly efficient gradient boosting decision tree. In NeurIPS, 2017.

[14] M. P. Naeini, G. Cooper, and M. Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning. In AAAI, 2015.

[15] G. W. Brier. Verification of forecasts expressed in terms of probability. Monthly Weather Review, 78(1):1–3, 1950.

[16] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. Science, 220(4598):671–680, 1983.

[17] Group 54. nsysu-youbike-log: logger, analysis code and results, github.com/alexlin8888/nsysu-youbike-log, 2026.